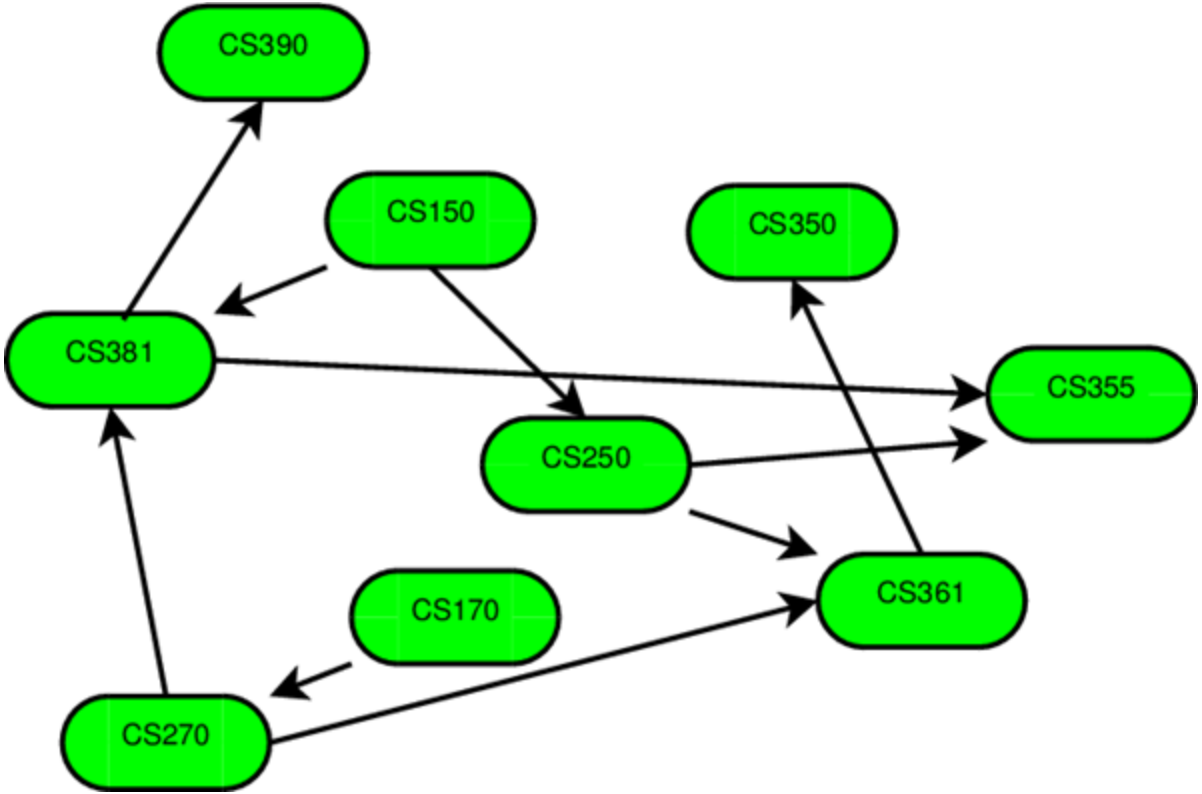
Graph Algorithms

Topological Sorting

- **Input:**
  - A Directed Acyclic Graph (DAG), G
- **Output:**
  - a linear ordering of all vertices such that if (u,v) is an edge in G, then u appears before v in the ordering
    - Linear ordering is called topological ordering
- Application: prerequisite among courses

Topological Sorting: Application



Question: how to verify a graph is DAG? Using DFS

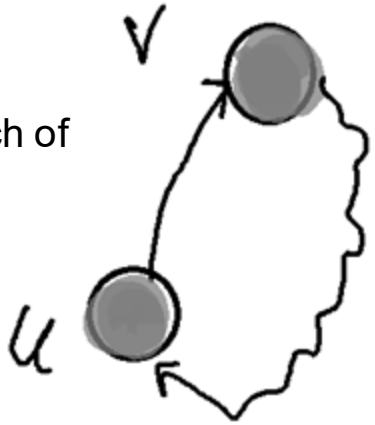
Applications of DFS

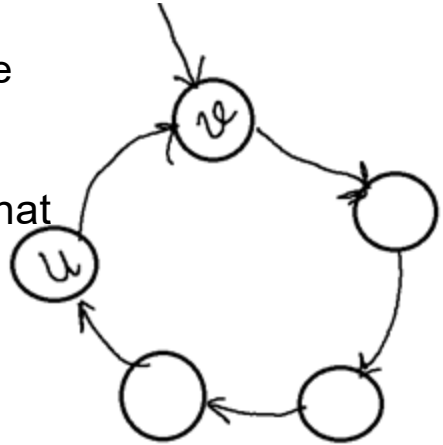
Detecting cycles in directed graphs

- **Lemma 1:** A graph G is acyclic if and only if a depth-first search of G yields no back edges
- **Equivalent to Lemma.** A directed graph has a (directed) cycle iff DFS has a back edge.

Detecting Cycles in graphs

- **Lemma 1:** A graph G is acyclic if and only if a depth-first search of G yields no back edges
- **Proof** ⇒ By contradiction.
  - Suppose DFS generates a back edge (u, v).
    - Therefore, v is an ancestor of u in the DFS tree
    - There is a path from v to u and the edge (u,v) completes the cycle→contradiction: We assumed G is acyclic
  - ⇐ By contradiction. Suppose G contains a cycle
    - **v**: the first vertex to be discovered in the cycle
    - All the vertices on the cycle must be discovered before we finish v. When we test edge (u,v), it will be a back edge
    - **(u,v)** is a back edge → contradicts the assumption that DFS of G yields no back edge

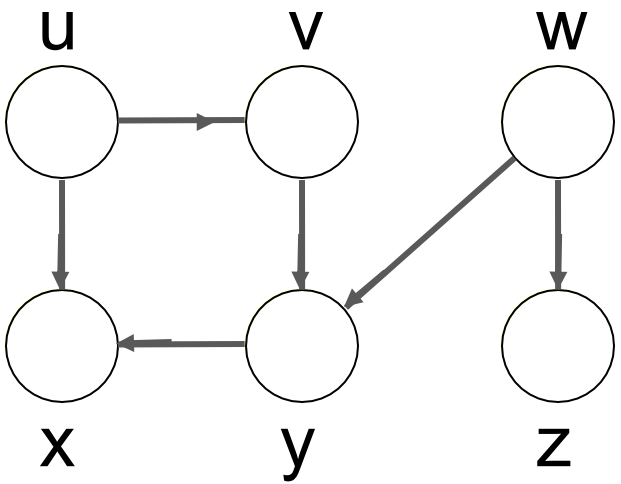




Topological Sorting: First Algorithm

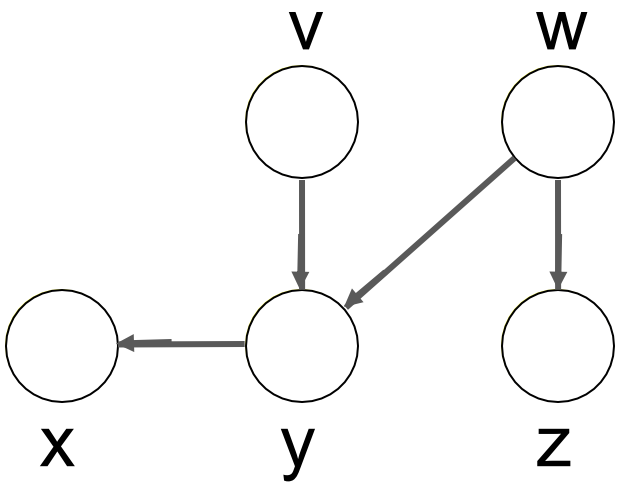
- repeatedly find a node with no incoming edges, remove it, and add it to the result
  - This algorithm is discussed in Exercise 22.4-5 of CLR (3rd edition)

Topological Sorting



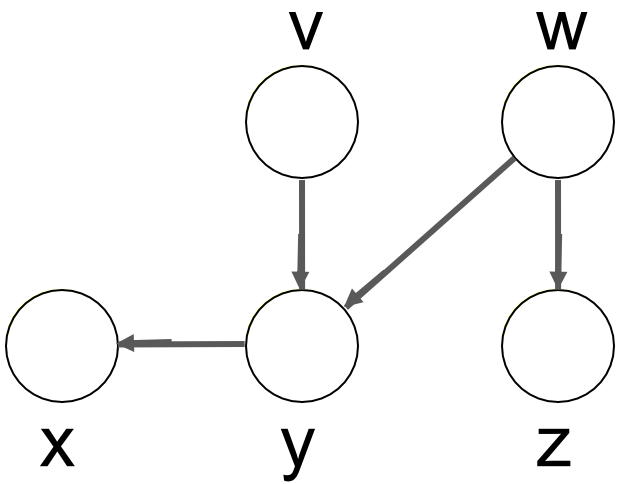
Ordered list:

Topological Sorting



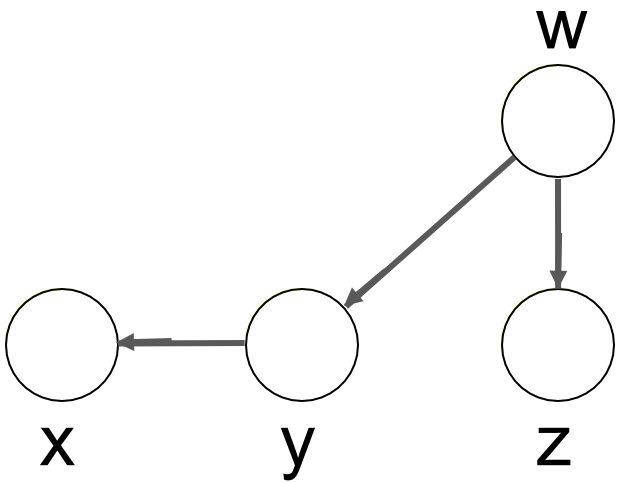
Ordered list: **u**

Topological Sorting



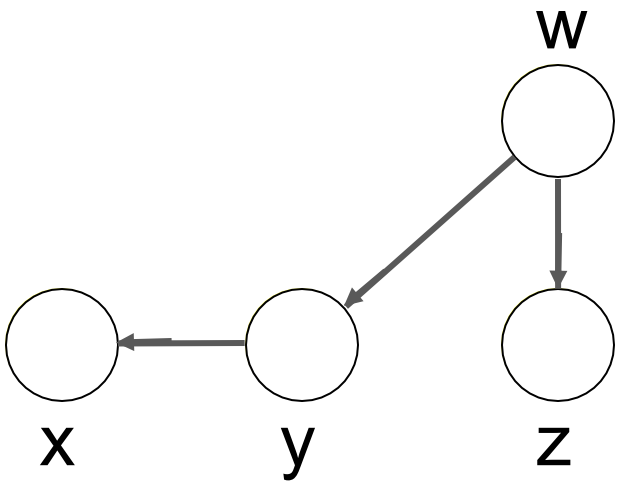
Ordered list: **u v**

Topological Sorting



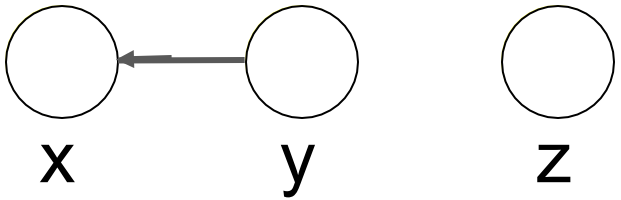
Ordered list: **u v**

Topological Sorting



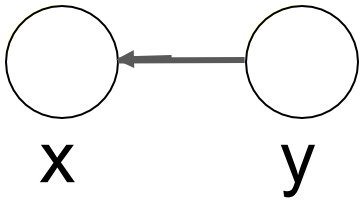
Ordered list: **u v w**

Topological Sorting



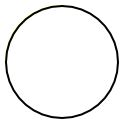
Ordered list: **u v w**

Topological Sorting



Ordered list: **u v w z**

Topological Sorting



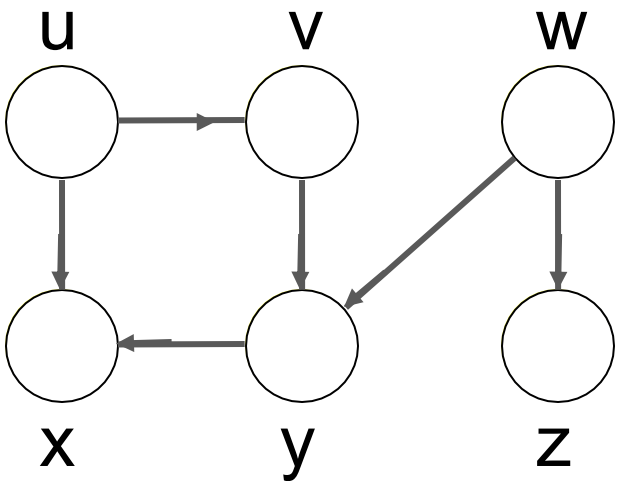
X

Ordered list: **u v w z y**

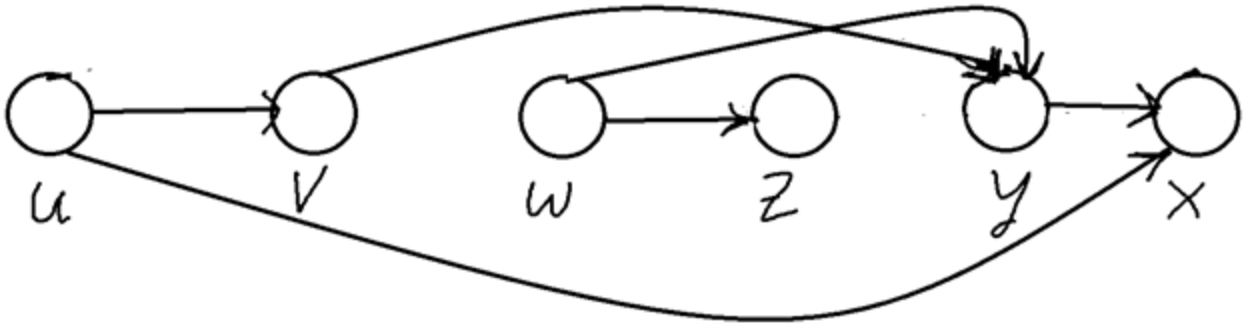
Topological Sorting

Ordered list: **u v w z y x**

Topological Sorting



Ordered list: **u v w z y x**



Topological Sorting: First Algorithm: Runtime

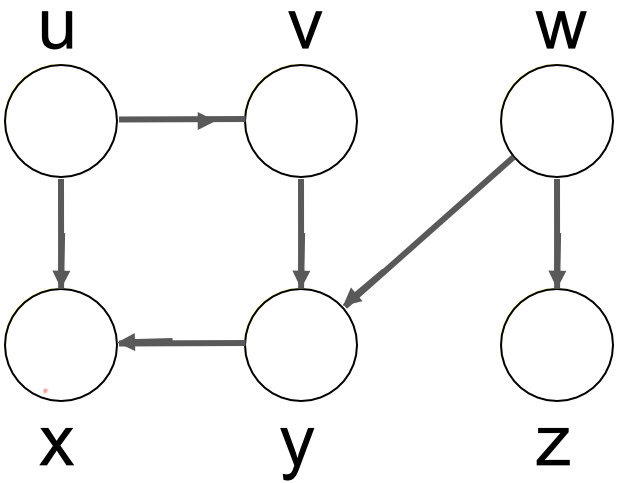
```
def topologicalSort-FirstAlg(G):
    result=[]
    while G is not empty:
        v=a vertex in G with indegree 0
        add v to result
        remove v and its edges from G
    return result
```

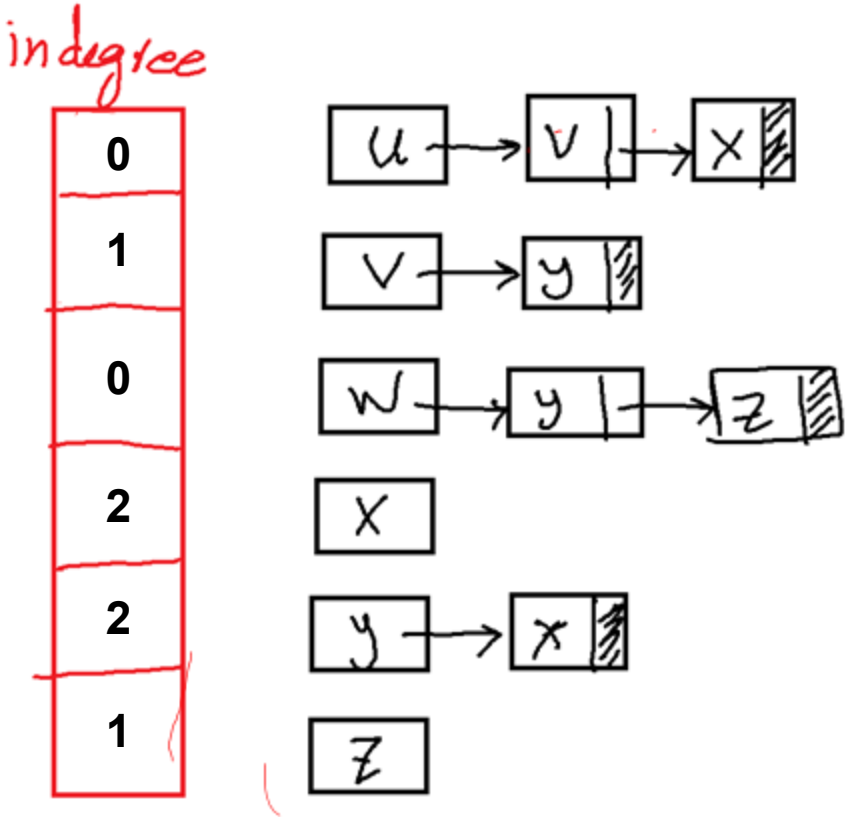
Runtime: $O(V^2+E)$

Can we do better?

- By changing the way finding for each vertex with indegree 0 is done
- Use a queue/stack to keep track of vertices with indegree 0
- Runtime: $\Theta(V + E)$

Topological Sorting: First Algorithm: Runtime





Question

How to initialize indegree array?

Question

How to use stack/queue?

Topological Sorting: An algorithm Based on DFS

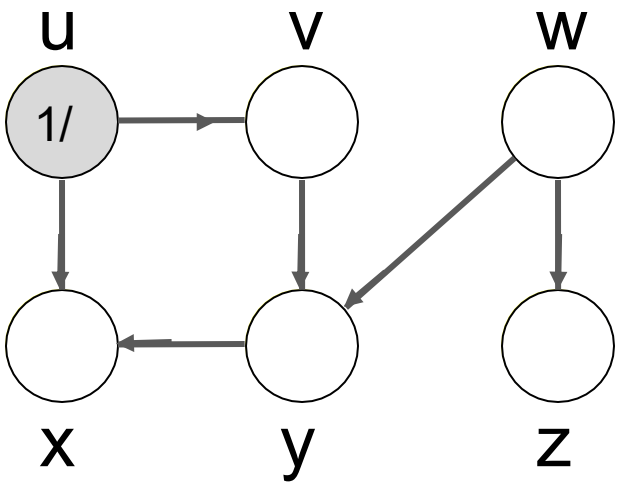
```
def TopologicalSort(G):  #G must be a DAG
        Run DFS(G) to compute finish[v] for all v ∈ V
        Output the vertices in decreasing order of their finish time
        return the linked list
```

Topological Sorting: An algorithm Based on DFS

```
DFS_ALL(G)
        for each vertex u in G:
                Color[u] = WHITE
                Pred[u] = NULL
        time =0
        oderedList=[]
        for each vertex u in G:
                if color[u] == WHITE
                        DFS(G, u, orderedList)
DFS(G, u, orderedList)
         time=time+1
        discover[u] = time
        color[u] = GRAY
        for each neighbor v of u
                 if (color[v] == WHITE)
                        pred[v] = u
                        DFS(G, v)
                color[u] = BLACK
                orderedList.insertfront(u)
                time = time + 1
        finish[u] = time
```

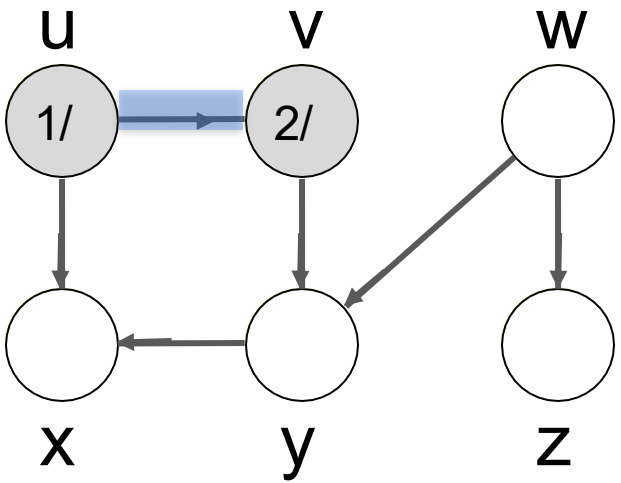
Runtime Analysis:
Θ(V+E)

Topological Sorting: An algorithm Based on DFS



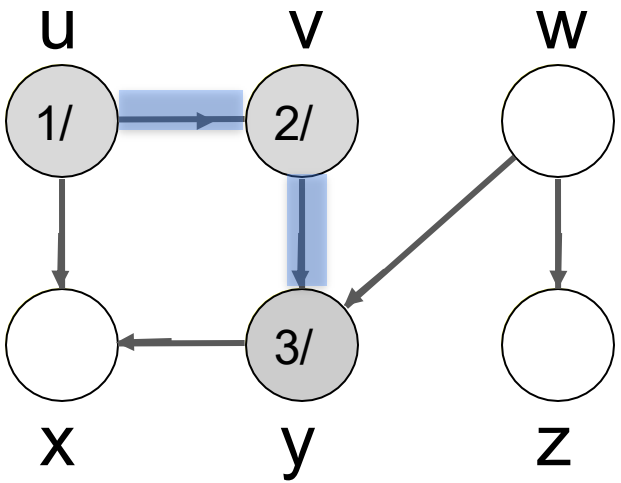
Ordered list:

Topological Sorting: An algorithm Based on DFS



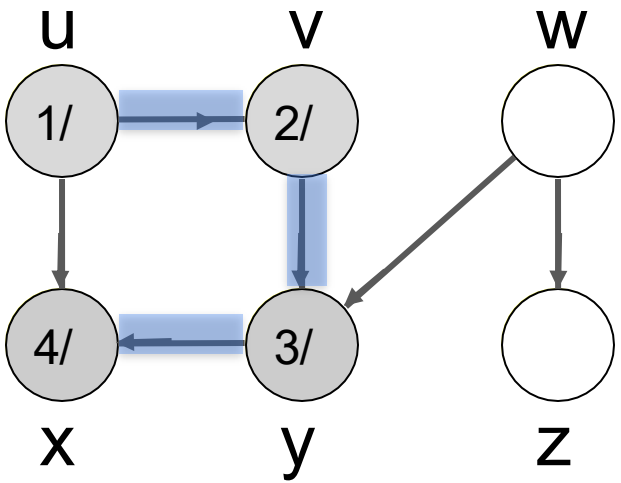
Ordered list:

Topological Sorting: An algorithm Based on DFS



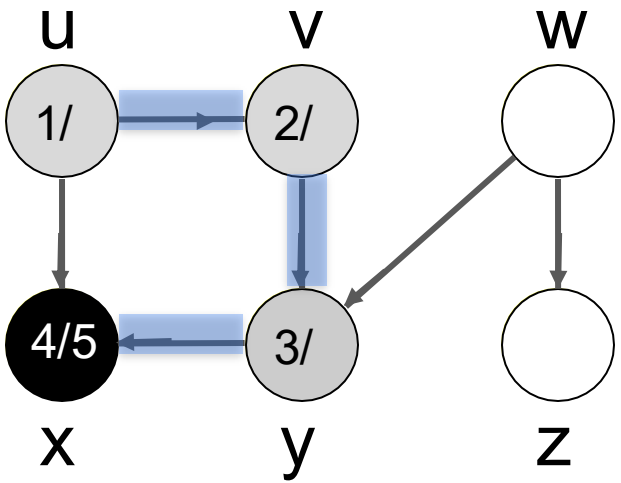
Ordered list:

Topological Sorting: An algorithm Based on DFS



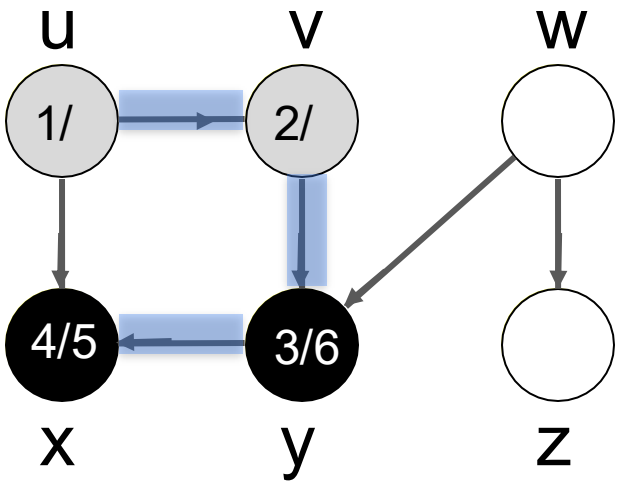
Ordered list:

Topological Sorting: An algorithm Based on DFS



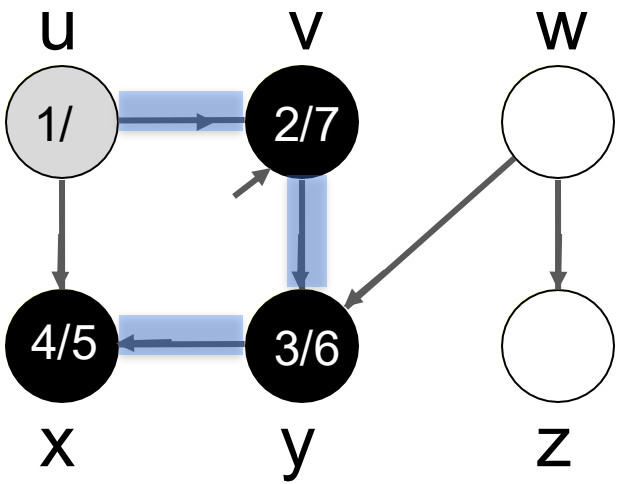
Ordered list: **x**

Topological Sorting: An algorithm Based on DFS



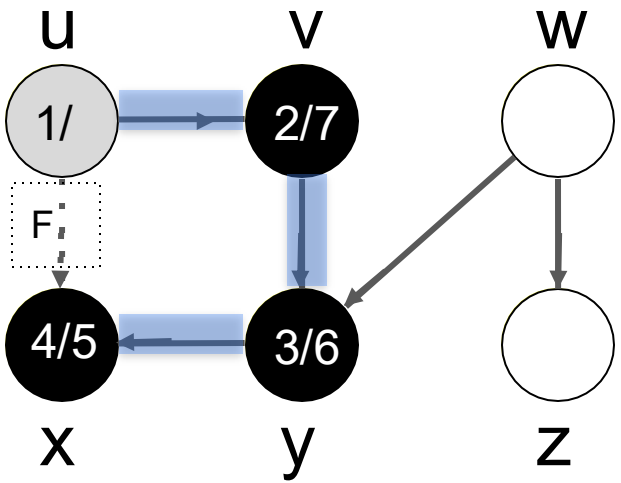
Ordered list: **y x**

Topological Sorting: An algorithm Based on DFS



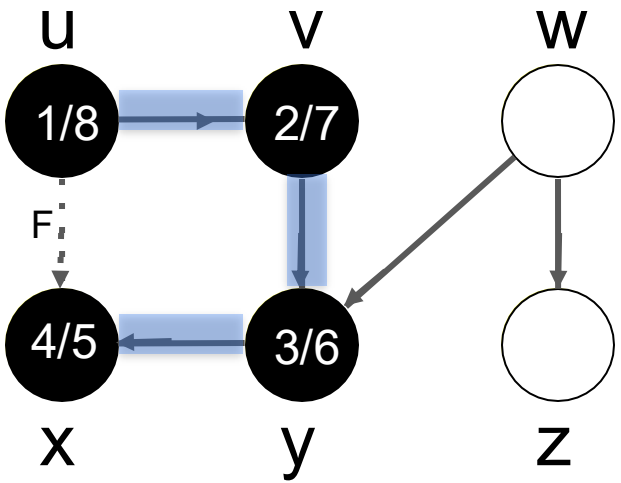
Ordered list: **v y x**

Topological Sorting: An algorithm Based on DFS



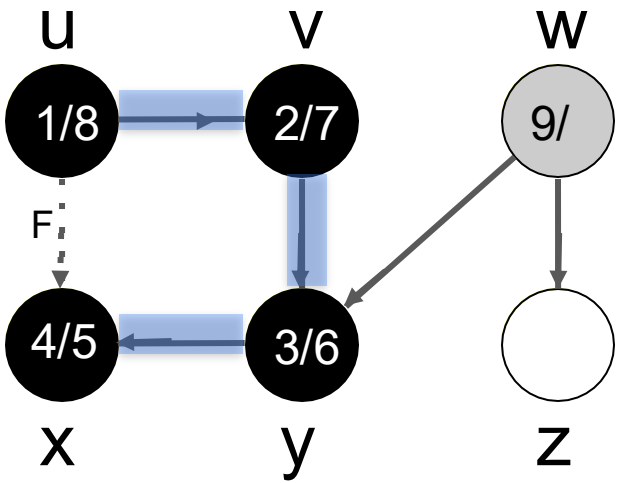
Ordered list: **v y x**

Topological Sorting: An algorithm Based on DFS



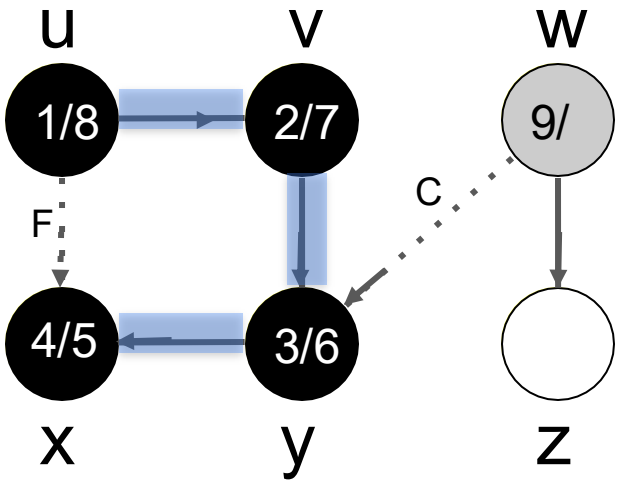
Ordered list: **u v y x**

Topological Sorting: An algorithm Based on DFS



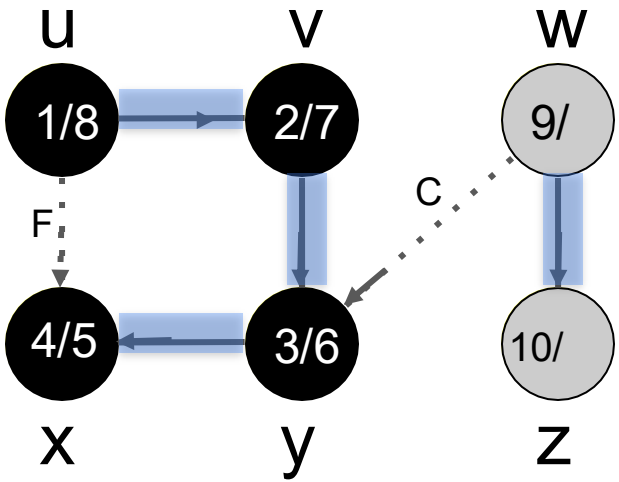
Ordered list: **u v y x**

Topological Sorting: An algorithm Based on DFS



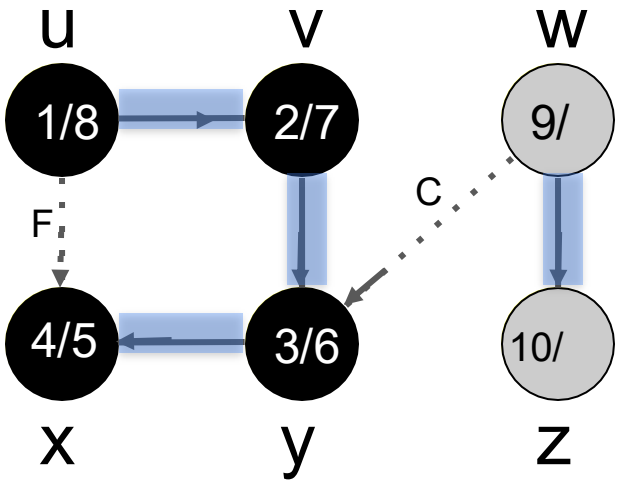
Ordered list: **u v y x**

Topological Sorting: An algorithm Based on DFS



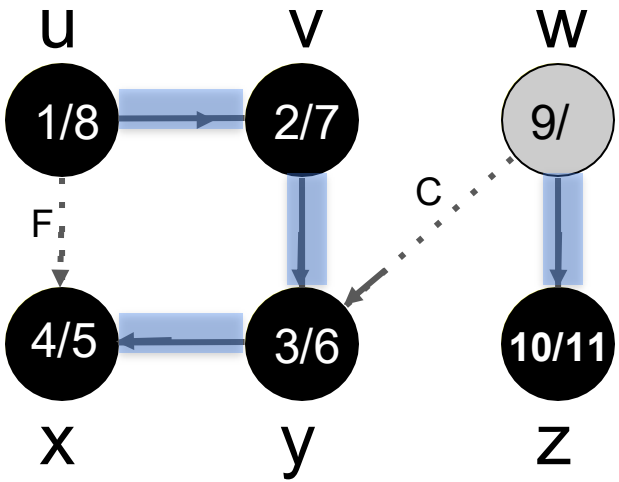
Ordered list: **u v y x**

Topological Sorting: An algorithm Based on DFS



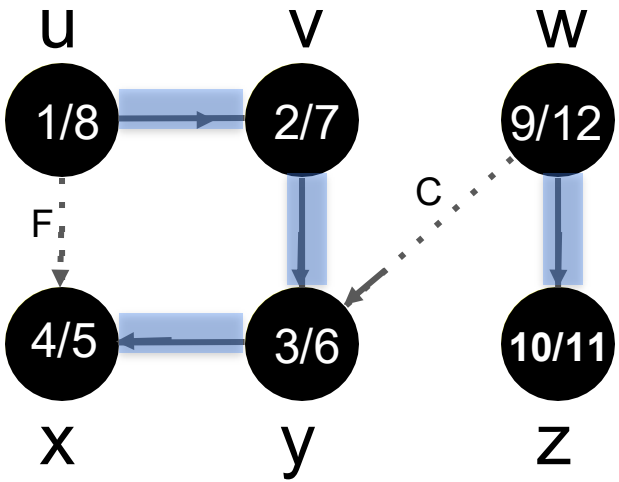
Ordered list: **u v y x**

Topological Sorting: An algorithm Based on DFS



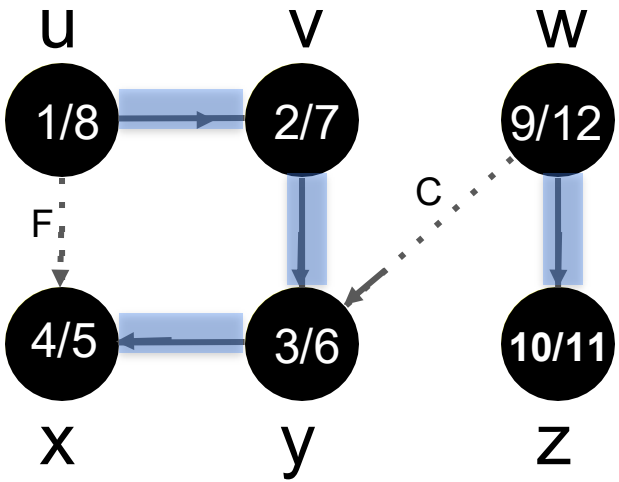
Ordered list: **z u v y x**

Topological Sorting: An algorithm Based on DFS



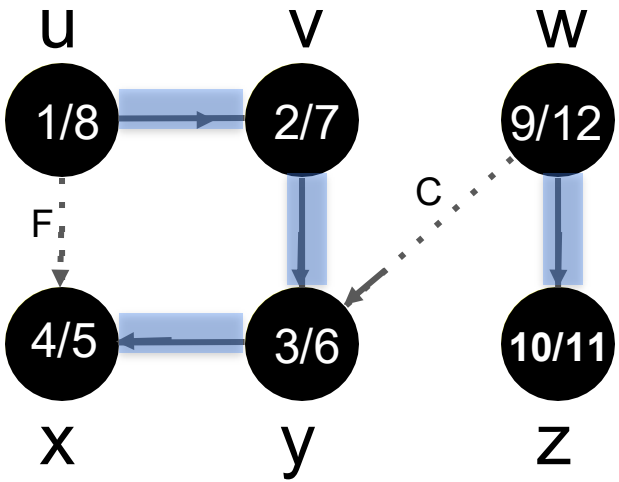
Ordered list: **w z u v y x**

Topological Sorting: An algorithm Based on DFS



Ordered list: **w z u v y x**

Topological Sorting: An algorithm Based on DFS



Ordered list: **w z u v y x**

